Datetime formating

To change the date format, we use strftime (string format time), for more information visit this website

لتغيير تنسيق التاريخ نستخدم strftime (string format time) لمزيد من المعلومات زور هذه الصفحه

https://www.w3schools.com/python/gloss_python_date_format_codes.asp

مثال:

```
from datetime import datetime

date = datetime(year=2026, month=2, day=22, hour=9, minute=30,
second=11)

print(date.strftime("%A %m/%d %I:%M %p"))
# output: Sunday 02/22 09:30 AM
```

To convert a str to a datetime object we use strptime (string parse time)

لتحويل النصوص الى datetime نستخدم strptime (string parse time)

```
from datetime import datetime

date = datetime.strptime("2026/12/8", "%Y/%m/%d")
print(date)
# output: 2026-12-08 00:00:00
```

Type Hints:

Type hints are very important to avoid using bad types and avoid errors, let's see an example with dict

الـ Type Hints مهمه جداً لتفادى الأخطاء سواء كانت املائيه او أنواع خاطئه، لنرى مثال مع الـ dict

```
data: list[dict] = [
    {"name":"alice", "age":20},
    {"name": 'bob', "aeg": 25},
]
print(data)
```

in this case we might do anything wrong even when we have big nasted data it will be confusing, let's see a better approach

في هذه الحاله يمكننا ان نقوم بأخطاء وعندما يكون لدينا بيانات كثيره داخل بعضها ممكن ان يكون الموضوع صعب ومحير، لنرى مثال أفضل

```
from typing import TypedDict

class Person(TypedDict):
    name: str
    age: int


data: list[Person] = [
    Person(name="alice", age=20),
    # or
    {"name": 'bob', "age": 25},
]
print(data)
```

in this way, we will always be notified when we do something wrong

بهذه الطريقه، سنعرف دائماً اذا فعلنا شيءً خاطئ

dataclass:

the dataclass is a decorator used for classes to tell python that this class is meant to store data, let's see a bad example

الـ dataclass هو decorator يستخدم للـ class لنعرف بايثون ان هذا class سيستخدم لتخزين البيانات، لنرى مثال

```
class Person:
    def __init__(self, name: str, age: int, country: str) -> None:
        self.name: str = name
        self.age: int = age
        self.country: str = country

    def __eq__(self, value) -> bool:
        return self.name == value.name and self.age == value.age

    def __gt__(self, other) -> bool: # >
        return self.age > other.age

    def __lt__(self, other) -> bool: # <
        return self.age < other.age

    def __le__(self, other) -> bool: # <=
        return self.age <= other.age

    def __repr__(self) -> str:
        return f"Person(name={self.name}, age={self.age}, country={self.country})"

data: list[Person] = [
    Person("alice", 20, 'USA'),
    Person("alice", 20, 'USA'),
    Person('bob', 25, 'UK')
]
print(data)
print(data[0] == data[1]) # True
print(data[0] < data[2]) # True
print(data[2] > data[0]) # True
```

as you can see, we wrote a lot of code, and if we want more data, it would be a waste of time, a better approach

كما ترى قمنا بكتابه الكثير من الاكواد، ان كنا نريد ان نقوم بإضافة المزيد سوف يكون مضيعة للوقت، مثال أفضل

```
from dataclasses import dataclass

@dataclass
class Person:
    name: str
    age: int
    country: str

    def __gt__(self, other) -> bool: # >
        return self.age > other.age

    def __lt__(self, other) -> bool: # <
        return self.age < other.age

    def __le__(self, other) -> bool: # <=
        return self.age <= other.age

data: list[Person] = [
    Person("alice", 20, 'USA'),
    Person("alice", 20, 'USA'),
    Person('bob', 25, 'UK')
]

print(data[0] == data[1]) # True
print(data[0] < data[2]) # True
print(data[2] > data[0]) # True
```

as you can see, we wrote a less code and its way better

كما ترى قمنا بكتابه اكواد اقل بكثير وبشكل أفضل